Implementing Conditions

In this homework your will implement some moderately complex conditions in MIPS assembly language. Please work on the problems yourself for as long as possible. If you get stuck, please seek help from peers or from your instructor.

Exercises

1. Implement the following pseudo-code in MIPS assembly language.

```
/* 1. Get an integer value from keyboard and store it in variable x */
/* 2. Get an integer value from keyboard and store it in variable y */
/* 3. Get an integer value from keyboard and store it in variable z */
   if ( ( x < 10 ) || ( ( y != 4 ) && ( z > 2 ) ) {
      print on console "I am feeling good.\n"
   } else {
      print on console "I am feeling drowsy.\n"
   }
   print on console "End of program\n"
```

2. Implement the following pseudo-code in MIPS assembly language.

```
/* 1. Get an integer value from keyboard and store it in variable a */
/* 2. Get an integer value from keyboard and store it in variable b */
/* 3. Get an integer value from keyboard and store it in variable c */
   if ( ( a < 10 ) && ( a > -3 ) ) {
      if ( ( b >= 0 ) && ( c <= 5 ) {
         print on console "You are my friend.\n"
      }
   } else {
      if ( ( b < 0 ) && ( c > 5 ) {
         print on console "You are my best friend.\n"
      } else {
         print on console "Ricky Ponting is my favorite cricketer.\n"
      }
   }
   print on console "End of program\n"
```

3. Implement the following pseudo-code in MIPS assembly language.

```
/* 1. Get an 32-bit float value from keyboard and store it in variable u     */
/* 2. Get a byte or character value from keyboard and store it in variable v */
/* 3. Get an integer value from keyboard and store it in variable w          */
   if ( ( u < 9.8 ) && ( v == 'm' ) ) {
      print on console "You live on Moon.\n"
   } else {
      if ( ( u < 1.0 ) && ( w >= 1 ) {
         print on console "You probably live on Pluto.\n"
      } else {
         print on console "You are an alien visiting Earth.\n"
      }
   }
   print on console "End of program\n"
```

Note

I would suggest using the MIPS general purpose registers as variables. As you know, in assembly language as well as high-level languages, variables are memory locations. Since MIPS has load-store architecture, the values of variables are first brought in registers and then operations are performed using them.

Submission Instructions

Solve each problem in a separate assembly language source file. Submit the following files on Google Classroom.

1. `<rollnumber>-h1-ex1.asm` containing solution of exercise 1.
2. `<rollnumber>-h1-ex2.asm` containing solution of exercise 2.
3. `<rollnumber>-h1-ex3.asm` containing solution of exercise 3.

Strictly following the above naming convention. You will certainly lose marks otherwise.